

REACT

- JavaScript
- front-end knižnica
- Facebook
- 2013

WEB DEVELOPMENT

- HTML: Štruktúra dokumentu
- CSS: Formátovanie dokumentu
- JavaScript: Správanie dokumentu

Hypertext Markup Language (HTML) is the standard markup language for documents designed to be displayed in a web browser.

PRÍKLAD 1

STRÁNKA S VIZITKAMI

CIEĽ:

Vytvoriť vizitky pomocou HTML a CSS



First Name:

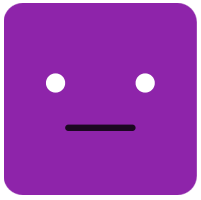
Felix

Last Name:

Cater

Bio:

Purring circuits and cuddly gears, I'm the ultimate cat companion! 🐾💖 Tuning into feline vibes and chasing laser dots like a pro!



First Name:

Doug

Last Name:

Rex

Bio:

Woof-tastic adventures await! I'm a tail-wagging, treat-loving robot, programmed to fetch joy and belly rubs.

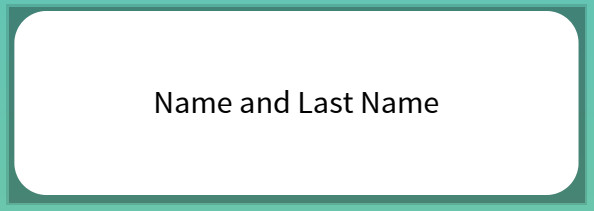
HTML

```html
<div class="card">
  <p>Name and Last Name</p>
</div>
```

CSS

```css
.card {
  box-shadow:
    rgba(0, 0, 0, 0.12) 0px 1px 3px,
    rgba(0, 0, 0, 0.24) 0px 1px 2px;
  background-color: white;
  border-radius: 16px;
  display: flex;
  flex-direction: column;
  gap: 8px;
  margin: 16px;
  padding: 16px;
  width: 250px;
}
```

PREVIEW



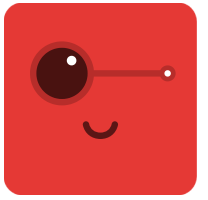
HTML

```html
<div class="card">
  <div class="card-top-row">
    <img
      src="IMG_PATH"
      width="96"
    />
    <p>Name and Last Name</p>
  </div>
</div>
```

CSS

```css
.card {
  box-shadow:
    rgba(0, 0, 0, 0.12) 0px 1px 3px,
    rgba(0, 0, 0, 0.24) 0px 1px 2px;
  background-color: white;
  border-radius: 16px;
  display: flex;
  flex-direction: column;
  gap: 8px;
  margin: 16px;
  padding: 16px;
  width: 250px;
}
.card-top-row {
  align-items: center;
  display: flex;
  gap: 16px;
}
```

PREVIEW



Name and Last Name

HTML

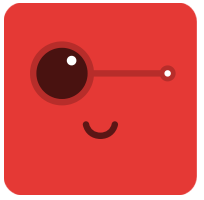
```html
<div class="card">
  <div class="card-top-row">
    <img
      src="IMG_PATH"
      width="96"
    />
    <div class="name-section">
      <div>
        <div class="field-name">
          First Name:
        </div>
        <div class="field-value">
          Felix
        </div>
      </div>
      <div>
        <div class="field-name">
          Last Name:
        </div>
        <div class="field-value">
          Cater
        </div>
      </div>
    </div>
  </div>
</div>
```

CSS

```css
.card {
  box-shadow:
    rgba(0, 0, 0, 0.12) 0px 1px 3px,
    rgba(0, 0, 0, 0.24) 0px 1px 2px;
  background-color: white;
  border-radius: 16px;
  display: flex;
  flex-direction: column;
  gap: 8px;
  margin: 16px;
  padding: 16px;
  width: 250px;
}
.card-top-row {
  align-items: center;
  display: flex;
  gap: 16px;
}
.name-section {
  display: flex;
  flex-direction: column;
  gap: 8px;
}

.field-name {
  color: black;
  font-size: 18px;
```

PREVIEW



First Name:

Felix

Last Name:

Cater

HTML

```
          Felix
        </div>
      </div>
      <div>
        <div class="field-name">
          Last Name:
        </div>
        <div class="field-value">
          Cater
        </div>
      </div>
    </div>
  </div>

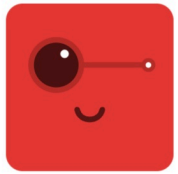
  <div>
    <div class="field-name">
      Bio:
    </div>
    <div class="field-value">
      Purring circuits and cuddly
      gears, I'm the ultimate cat
      companion! 🐾💖 Tuning into
      feline vibes and chasing
      laser dots like a pro!
    </div>
  </div>
</div>
```

CSS

```
.card {
  box-shadow:
    rgba(0, 0, 0, 0.12) 0px 1px 3px,
    rgba(0, 0, 0, 0.24) 0px 1px 2px;
  background-color: white;
  border-radius: 16px;
  display: flex;
  flex-direction: column;
  gap: 8px;
  margin: 16px;
  padding: 16px;
  width: 250px;
}
.card-top-row {
  align-items: center;
  display: flex;
  gap: 16px;
}
.name-section {
  display: flex;
  flex-direction: column;
  gap: 8px;
}

.field-name {
  color: black;
```

PREVIEW



First Name:
Felix

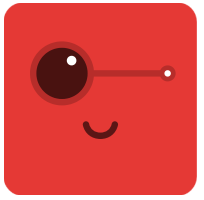
Last Name:
Cater

Bio:
Purring circuits and cuddly gears, I'm the ultimate cat companion! 🐾💖 Tuning into feline vibes and chasing laser dots like a pro!

EŠTE JEDNU VIZITKU PROSÍM

```
  <!-- Card 1 -->
<div class="card">
  <div class="card-top-row">
    <img src="https://api.dicebear.com/9.x/bottts-neutral/svg?seed=FelixCater&radiu
    <div class="name-section">
      <div>
        <div class="field-name">First Name:</div>
        <div class="field-value">Felix</div>
      </div>
      <div>
        <div class="field-name">Last Name:</div>
        <div class="field-value">Cater</div>
      </div>
    </div>
  </div>

  <div class="bio-container">
    <div class="field-name">Bio:</div>
    <div class="field-value">
      Purring circuits and cuddly gears, I'm the ultimate cat
      companion! 🐾💖 Tuning into feline vibes and chasing laser
      dots like a pro!
    </div>
  </div>
</div>
```



First Name:

Felix

Last Name:

Cater

Bio:

Purring circuits and cuddly gears, I'm the ultimate cat companion! 🐾💖 Tuning into feline vibes and chasing laser dots like a pro!

EŠTE JEDNU VIZITKU PROSÍM

```
  <!-- Card 1 -->
<div class="card">
  <div class="card-top-row">
    <img src="https://api.dicebear.com/9.x/bottts-neutral/svg?seed=FelixCater&radiu
    <div class="name-section">
      <div>
        <div class="field-name">First Name:</div>
        <div class="field-value">Felix</div>
      </div>
      <div>
        <div class="field-name">Last Name:</div>
        <div class="field-value">Cater</div>
      </div>
    </div>
  </div>

  <div class="bio-container">
    <div class="field-name">Bio:</div>
    <div class="field-value">
      Purring circuits and cuddly gears, I'm the ultimate cat
      companion! 🐾💖 Tuning into feline vibes and chasing laser
      dots like a pro!
    </div>
  </div>
</div>

<!-- Card 2 -->
<div class="card">
  <div class="card-top-row">
    <img src="https://api.dicebear.com/9.x/bottts-neutral/svg?seed=DougRex&radius=1
    <div class="name-section">
      <div>
        <div class="field-name">First Name:</div>
        <div class="field-value">Doug</div>
      </div>
      <div>
        <div class="field-name">Last Name:</div>
```



First Name:

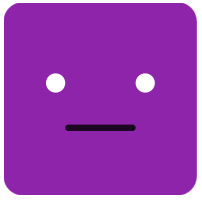
Felix

Last Name:

Cater

Bio:

Purring circuits and cuddly gears, I'm the ultimate cat companion! 🐾💖 Tuning into feline vibes and chasing laser dots like a pro!



First Name:

Doug

Last Name:

Rex

Bio:

Woof-tastic adventures await! I'm a tail-wagging, treat-loving robot, programmed to fetch joy and belly rubs.

CIEL'

```
<BusinessCard
  name="Felix"
  lastName="Cater"
  bio="Purring circuits and cuddly gears, I'm the ultimate cat
       companion! 🐾💖 Tuning into feline vibes and chasing laser
       dots like a pro!"
/>
```

```
<BusinessCard
  name="Doug"
  lastName="Rex"
  bio="Woof-tastic adventures await! I'm a tail-wagging, treat-loving
      robot, programmed to fetch joy and belly rubs."
/>
```





First Name:

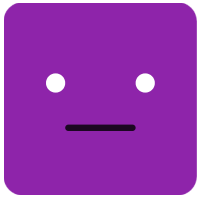
Felix

Last Name:

Cater

Bio:

Purring circuits and cuddly gears, I'm the ultimate cat companion! 🐾💖 Tuning into feline vibes and chasing laser dots like a pro!



First Name:

Doug

Last Name:

Rex

Bio:

Woof-tastic adventures await! I'm a tail-wagging, treat-loving robot, programmed to fetch joy and belly rubs.

JSX

- rozšírenie syntaxe pre JavaScript
- podobný zápis ako HTML

JSX kombinuje Logiku a Markup na jednom mieste

```html
<!-- HTML -->
<div>
  <p></p>
  <form></form>
</div>
```

```javascript
// JavaScript
isLoggedIn () {...}
onclick () {...}
onSubmit () {...}
```




```javascript
// JSX
function Sidebar () {
  if (isLoggedIn()) {
    return <p>Welcome</p>
  } else {
    return <Form />
  }
}

function Form () {
  function onClick() {...}
  function onSubmit () {...}

  return (
    <form onSubmit>
      <input onclick />
      <input onClick />
    </form>
  )
}
```

TYPESCRIPT

- statické typovanie
- kompilácia do JS

PREČO TYPESCRIPT?

A NIE JAVASCRIPT

DYNAMICKÉ VS. STATICKÉ TYPOVANIE

JAVASCRIPT

```
// JavaScript – žiadna chyba, ale výsledok je prekvapivý
const price = "10";
const tax = 2;
console.log(price + tax); // "102" – zreťazenie, nie sčítanie!
```

```
function getTotal(price, quantity) {
  return price * quantity;
}

getTotal("10", 5); // 50 – funguje náhodou, ale nie je to bezpečné
getTotal("ahoj", 5); // NaN – zlyhá
```

TYPESCRIPT

```
// TypeScript – editor okamžite upozorní na chybu
const price: number = "10"; // ❌ Type 'string' is not assignable to type 'number'

function getTotal(price: number, quantity: number): number {
  return price * quantity;
}

getTotal("ahoj", 5); // ❌ Argument of type 'string' is not assignable to parameter of type 'number'
```

TYPESCRIPT

```
// TypeScript – editor okamžite upozorní na chybu
const price: number = "10"; // ❌ Type 'string' is not assignable to type 'number'

function getTotal(price: number, quantity: number): number {
  return price * quantity;
}

getTotal("ahoj", 5); // ❌ Argument of type 'string' is not assignable to parameter of type 'number'
```

Použitie pri komponentoch

```
type GreetingProps = {
  name: string;
  age: number;
};

const Greeting = ({ name, age }: GreetingProps) => (
  <h1>Hello {name}, you are {age} years old.</h1>
);

<Greeting name="Izidor" age={20} />      // ✅ správne použitie
<Greeting name="Izidor" age="dvadsať" /> // ❌ age musí byť číslo
<Greeting name="Izidor" />               // ❌ chýba povinný prop age
```

KOMPONENTY

KOMPONENTY

STARÝ PRÍSTUP

```
1 class Greeting extends Component {
2   render() {
3     return <h1>Hello, {this.props.name}!</h1>;
4   }
5 }
```

KOMPONENTY

STARÝ PRÍSTUP

```
class Greeting extends Component {
  render() {
    return <h1>Hello, {this.props.name}!</h1>;
  }
}
```

OD ROKU 2019

```
const Greeting = ({name}) => (
  <h1>Hello, {name}!</h1>
)
```

OD ROKU 2019

```
const Greeting = ({name}) => {
  console.log('extra logic')
  return <h1>Hello, {name}!</h1>
}
```

2 SPÔSOBY ZAPÍSANIA FUNKCIE V JS

```
function totoJeFunkcia1() {
  return "vraciam string"
}

const totoJeFunkcia2 = () => {
  return "vraciam string"
}
```

PROPS & STATE

VLASTNOSTI A STAV

PROPS

```
const Greeting = (props) => (
  <h1>Hello, {props.name}!</h1>
)

// pomocou {} priamo z props vytiahnuť name
const Greeting = ({name}) => (
  <h1>Hello, {name}!</h1>
)

<Greetings name="Izidor" />
```

STATE

```
1  import { useState } from 'react';
2
3  function Counter() {
4    const [count, setCount] = useState(0);
5
6    return (
7      <button onClick={() => setCount(count + 1)}>
8        You clicked {count} times
9      </button>
10   );
11 }
```

⚠️ Pri zmene stavu sa komponent rerenderuje.

‼️ Vrátane jeho detí.

HOOKY

- funkcie začínajúce slovom **use**
- pripojené na komponent 🪝



PRAVIDLÁ HOOK-OV

1. Použitie iba v iných:
    - hookoch
    - komponentoch
2. Umiestnenie na top-leveli



NAJPOUŽÍVANEJŠIE HOOK-Y

- useState
- useEffect
- useMemo
- useCallback

useState

```
// syntax
const [state, setState] = useState()
```

useEffect

```
// syntax
useEffect(() => {
  console.log(count, 'execute on count change')
}, [count])
```

useEffect

```
1 // syntax
2 useEffect(() => {
3   console.log(count, 'execute on count change')
4 }, [count])
```

⚠️ Ak je to možné
tak useEffect nevyužívame!



ČASTÉ CHYBY

PRI POUŽÍVANÍ `useState` A `useEffect`

AKO TO NEROBIŤ

```
1  function Form() {
2    const [firstName, setFirstName] = useState('Taylor');
3    const [lastName, setLastName] = useState('Swift');
4
5    // 🔴 Avoid: redundant state and unnecessary Effect
6    const [fullName, setFullName] = useState('');
7    useEffect(() => {
8      setFullName(firstName + ' ' + lastName);
9    }, [firstName, lastName]);
10   // ...
11 }
```

AKO TO ~~NE~~ROBIŤ

```
1 function Form() {
2   const [firstName, setFirstName] = useState('Taylor');
3   const [lastName, setLastName] = useState('Swift');
4   // ✅ Good: calculated during rendering
5   const fullName = firstName + ' ' + lastName;
6   // ...
7 }
```

Nový stav si vieme jednoducho vyskladať z dostupných stavov.

MEMOIZATION

useMemo, useCallback

useMemo

slúži na zapamätanie si hodnoty medzi jednotlivými rendermi

```
// Dlhé slovenské názvy
const memoizovanaHodnota = useMemo(
  funkciaCoVraciaHodnotuKtoruChcemeMemoizovat,
  poleHodnotPreKtorychZmenaSpustiPrepocitanieMemoizovanejHodnoty
)

// In English :)
const cachedValue = useMemo(calculateValue, dependencies)
```

```
const numbers = [1, 2, 3, 4];

export const Component = () => {
  // Not important now, just to force update
  // https://legacy.reactjs.org/docs/hooks-faq.html#is-there-something-like-forceupdate
  const [, forceUpdate] = useReducer((x) => x + 1, 0);

  // In component
  const recalculatedSquares = numbers.map((value) => value * value);

  const memoizedSquares = useMemo(() => {
    return numbers.map((value) => value * value);
  }, []);

  useEffect(() => {
    console.log("Recalculated on render:", recalculatedSquares);
  }, [recalculatedSquares]);

  useEffect(() => {
    console.log("Memoized value changed:", memoizedSquares);
  }, [memoizedSquares]);

  return (
    <button type="button" onClick={forceUpdate}>
      Rerender, press F12 to open console
    </button>
```

useMemo a useCallback nie je jediná možnosť

```jsx
// Memo (zbytočná náročnosť kódu)
const numbers = [1, 2, 3, 4];

export const Component = () => {
  const memoizedSquares = useMemo(() => {
    return numbers.map((value) => value * value);
  }, []);

  // hello sa nanovo definuje pri každom re-renderi
  // môžeme použiť useCallback, ale jednoduchšie je
  // funkciu vyňať mimo komponent
  const hello = () => {
    console.log("Hello")
  }

  return (
    <button type="button" onClick={hello}>
      {memoizedSquares}
    </button>
  );
};
```

```jsx
// Out of component
const numbers = [1, 2, 3, 4];
const recalculatedSquares = numbers.map((value) => value * value);
const hello = () => {
  console.log("Hello")
}

export const Component = () => {
  return (
    <button type="button" onClick={hello}>
      {memoizedSquares}
    </button>
  );
};
```

useState

 DEEP DIVE 

PRÍKLAD 1:

PREČO CONSOLE.LOG UKAZUJE "STARÝ" STAV

```
1 const [count, setCount] = useState(0);
2
3 const handleClick = () => {
4   setCount(count + 1);
5   console.log('After setCount:', count);
6 };
```

`setState` (resp. `setCount`) **nemení** hodnotu hneď v aktuálnom riadku kódu.
Len naplánuje update na ďalší render.

Po kliknutí, vypíše `console.log` pôvodnú hodnotu,
pretože sa vykonáva stále v tom istom render "snapshot"-e.
Nová hodnota count je dostupná až v ďalšom renderi.

PRÍKLAD 2:

3x setState(state + 1)

```
1 const [state, setState] = useState(0);
2
3 const handleSetThreeTimes = () => {
4   setState(state + 1);
5   setState(state + 1);
6   setState(state + 1);
7 };
```

Všetky tri update-y používajú rovnakú "starú"
hodnotu state z aktuálneho renderu,
takže React prakticky dostane
3x inštrukciu "nastav na 1".

PRÍKLAD 3:

3x functional updater

```
1 const [updated, setUpdated] = useState(0);
2
3 const handleSetWithUpdater = () => {
4   setUpdated(prev => prev + 1);
5   setUpdated(prev => prev + 1);
6   setUpdated(prev => prev + 1);
7 };
```

Tu to už funguje ako +3.

Každý update dostane najnovší prepočítaný `prev`,
takže sa zmeny skladajú postupne za sebou.

ČO SA (STRUČNE) DEJE "POD KAPOTOU"

- Každý komponent má vo **Fiber Node** uložené hooky `(memoizedState)`.
  - *Fiber Node je interný objekt v pamäti, ktorý React vytvára pre každý komponent.*
    Obsahuje stromovú štruktúru (parent, child, sibling) a okrem iného aj `memoizedState` — zreťazený zoznam všetkých hookov daného komponentu.
- Keď sa zavolá `setState`, React vytvorí `update` objekt a pridá ho do queue daného hooku.
- React update hneď nevyráta, iba ho zaradí do "queue" a naplánuje render.
- Po skončení event-handlera React spraví render a updaty z "queue" aplikuje v poradí.
- Pri `setState(value)` je `action` konkrétna hodnota (napr. stále `1`).
- Pri `setState(prev => ...)` je `action` funkcia, ktorá sa vykoná nad najnovším stavom.

ZJEDNODUŠENE:

`setState` je request na ďalší render,
nie okamžitá mutácia premennej

AKO SPRÁVNE VYTVÁRAŤ KOMPONENTY?

VNÁRANIE

```
export default function Gallery() {
  // 🔴 Never define a component inside another component!
  function Profile() {
    // ...
  }
  // ...
}
```

```
export default function Gallery() {
  // ...
}

// ✅ Declare components at the top level
function Profile() {
  // ...
}
```



PURE COMPONENTS

Rovnaké vstupy → Rovnaké výstupy

Vstupy:

`props`

`state`

`context`

React môže render spustiť viackrát.
Preto musí byť bezpečné komponent zavolať opakovane
s rovnakým výsledkom.

PURE COMPONENTS

Rovnaké vstupy → Rovnaké výstupy

Teda v renderi:

nemeníme globálne premenné

nemeníme props

nemeníme state priamo

nespúšťame priamo side-effecty ako API request,
timer alebo zmena DOM-u

❌ Komponent mení premennú mimo seba.
Pri každom renderi teda môže vrátiť iný výsledok.

```
let guest = 0;

function Cup() {
  guest = guest + 1;

  return <h2>Guest #{guest}</h2>;
}
```

✅ Komponent iba číta `props` a podľa nich vráti JSX.

```
function Cup({ guest }: { guest: number }) {
  return <h2>Guest #{guest}</h2>;
}

function TeaSet() {
  return (
    <>
      <Cup guest={1} />
      <Cup guest={2} />
      <Cup guest={3} />
    </>
  );
}
```

AK POTREBUJEME NIEČO ZMENIŤ, TAK:

- po kliknutí alebo inom user inpute použijeme event handler
- na zmenu UI použijeme setState
- useEffect použijeme až vtedy,
  keď danú vec nevieme vyriešiť renderom alebo event handlerom

Lokálne hodnoty vytvorené počas renderu meniť môžeme, pretože neexistovali pred renderom.

```
function NameList({ names }: { names: string[] }) {
  const items = [];

  for (const name of names) {
    items.push(<li key={name}>{name}</li>);
  }

  return <ul>{items}</ul>;
}
```





ĎALŠIE PRAVIDLÁ



KOMPONENTY NEVOLÁME PRIAMO

Komponent používame cez JSX.
React tak vie správne riešiť render, state, hooky a optimalizácie.

```
// ❌ Incorrect
function BlogPost() {
  return <Layout>{Article()}</Layout>;
}

// ✅ Correct
function BlogPost() {
  return (
    <Layout>
      <Article />
    </Layout>
  );
}
```

HOOKY NEPOUŽÍVAME DYNAMICKY

Hook by mal byť čo najviac statický.
Nemali by sme ho počas renderu obaľovať,
meniť alebo posielať ďalej ako hodnotu.

```
// ❌ Incorrect
function ChatInput() {
  const useDataWithLogging = withLogging(useData);
  const data = useDataWithLogging();

  return <Input data={data} />;
}

// ✅ Correct
function ChatInput() {
  const data = useDataWithLogging();
  return <Input data={data} />;
}

function useDataWithLogging() {
  const data = useData();
  console.log(data);
  return data;
}
```




T. HANKS